Model Kill-Switch Protocol

A stop that cannot be forged.

That cannot be turned against you.

And still works when everything around it falls apart.

The machine at a glance

Trust layer

- 1. Every command is signed (core)
- 4. A committee, not one person (core)
- 7. Guarding the guard

Gateway layer

- 3. The gateway (core)
- 5. Targeted stop
- 10. Simulate before stopping

Resilience layer

- 2. No second use
- 6. When the infrastructure falls
- 8. Many regions
- 11. Automatic expiry

Adoption layer

- 9. Developer libraries

Follow one command

1. A committee signs. Three of five committee members jointly produce one signature. A single key is not enough.

2. One signature. The command is signed (Ed25519, 64 bytes) together with a one-time number and a timestamp.

3. Spread to every region. The command is replicated across regions in the same order, even if links between them are slow.

4. The gateway checks. Four checks: signature, time (within 30 seconds), a number not seen before, and permission for the scope.

5. State updates. The state changes in each gateway's local memory, and checking every request takes constant time.

6. Client gets 503. New requests are blocked with an explanation. Requests already running finish, and are not cut mid-way.

7. Everything is logged. Every attempt is logged, and unusual use of a key raises an alert. The stop itself expires after four hours unless re-signed.

The features

Feature 1: Every command is signed

Only an authorised party can stop the model. Every command passes four checks before the gateway moves.

How it works

- The authority builds a command: action, scope, a one-time number and a timestamp.
- The command is signed with Ed25519. The signature is tiny, only 64 bytes.
- The gateway checks four things: signature, time (within 30 seconds), the number, and permission.
- Only if everything passes does the state change.

What it defends against

Threat: An attacker sends a command with a signature that does not match.

Answer: The command is rejected and logged as a suspicious attempt.

Technologies and design decisions

Component	Type	Role
PyNaCl / libsodium	Off the shelf	Sign and verify
HashiCorp Vault	Off the shelf	The private key does not sit in a file
HSM / Cloud KMS	Off the shelf	The key never leaves the hardware
mTLS + Envoy	Off the shelf	Mutual authentication on the network
Open Policy Agent	Off the shelf	Who may do what, as policy
The four-check chain	Assembled	Everything combined into one check

Data structure: CommandPayload

Field	Meaning
action	STOP or RESUME
scope	what it applies to
nonce	one-time number
timestamp	when it was signed
issuer_key_id	who signed

Input: A signed command

Output: Accepted or rejected, with a reason

Feature 2: No second use

A genuine command sent once will not work a second time.

How it works

- Every command carries a random 16-byte number.
- The gateway records the number in one atomic operation (SETNX), leaving no gap to exploit.
- If the number already exists, the command is rejected as a replay.
- The number is kept 60 seconds, matching the time window.

What it defends against

Threat: An attacker saves an old RESUME command and resends it to cancel a stop.

Answer: The number is already recorded, and the command is rejected at once.

Technologies and design decisions

Component	Type	Role
Redis (SETNX + TTL)	Off the shelf	Atomic check of the number
Redis Sentinel / Cluster	Off the shelf	Does not fall with a single node
Apache Kafka	Off the shelf	Same order on every server
Kubernetes NetworkPolicy	Off the shelf	A separate layer at network level
Prometheus + Grafana	Off the shelf	See how many replays were blocked
Number expiry = the time window	Our design decision	No need to keep numbers forever

Data structure: NonceRecord

Field	Meaning
key	nonce:‹number›
value	1 (the key existing is the information)
ttl	60 seconds

Input: The number from a command whose signature passed

Output: Replay or new

Feature 3: The gateway

Every request to the model gets a split-second decision: pass or block.

How it works

- The request arrives with a scope: which model, which client.
- The state is checked in local memory, in constant time, with no network call.
- Active: the request goes through. Suspended: a 503 with an explanation.
- Requests already running finish (up to 10 seconds). New ones do not enter.

What mistake it prevents

Threat: A hard stop that cuts an answer mid-way and leaves a partial, misleading reply.

Answer: Controlled drain: what started finishes, nothing new enters.

Technologies and design decisions

Component	Type	Role
Envoy Proxy + WASM	Off the shelf	The check runs inside the network layer
Istio Service Mesh	Off the shelf	Internal traffic honours the stop too
Circuit Breaker	Off the shelf	Early warning before a full stop
In-memory state updated from Kafka	Assembled	Constant-time check
Controlled drain	Our design decision	Answers are never cut

Data structure: GatewayState

Field	Meaning
status	ACTIVE or SUSPENDED
since	since when
reason	why
last_command_id	the command that caused it

Input: An HTTP request with scope details

Output: Pass to the model, or a 503 with an explanation

Feature 4: A committee, not one person

A valid command needs three of five signers. One stolen key is not enough.

How it works

- Five independent committee members, each holding a share of a key.
- Three of them jointly produce a single signature (FROST).
- The gateway checks one signature as usual. The complexity happens before sending.
- A suspect key is blocked at once on a revocation list. Key rotation overlaps by 24 hours.

What it defends against

Threat: One key is stolen, or one committee member is corrupt.

Answer: One signature is not enough, and the suspect key is blocked at once.

Technologies and design decisions

Component	Type	Role
FROST (threshold signature)	Off the shelf	M of N produce one signature
Vault Transit	Off the shelf	The key never leaves the service
CRL / OCSP	Off the shelf	Instant key revocation

Component	Type	Role
Transparent log inspired by Certificate Transparency	Our design decision	Anyone can see who holds stop authority
A committee instead of a single key	Our design decision	Answers "who guards the guards"

Data structure: AuthorizedSigner

Field	Meaning
key_id	key identifier
public_key_share	the member's key share
valid_from / valid_until	key validity
role	role, such as regulator

Input: Partial signatures from three members

Output: One signature, and an entry in the public log

Feature 5: Targeted stop

You do not always need to stop everything. One model, one region or one capability is possible.

How it works

- Every request is tagged: model, client, region and capabilities.
- The command carries a hierarchical scope, such as region:eu-*.
- A rule engine checks whether the request falls inside an active scope.
- If information is missing or corrupt, only that request is blocked and logged.

What mistake it prevents

Threat: A stop that is accidentally too broad, or missing information that confuses the gateway.

Answer: Fail closed at request level only, without harming anything else.

Technologies and design decisions

Component	Type	Role
OPA + Rego	Off the shelf	Wildcards and hierarchy in scopes
Inspired by LaunchDarkly	Off the shelf	Fast evaluation engine with a local cache
Bloom Filter	Off the shelf	Fast filtering at scale
Default: fail closed for the request	Our design decision	Doubt wins over convenience

Data structure: ScopeRule

Field	Meaning
pattern	model:gpt-x/capability:code-execution
status	SUSPENDED
created_by	who created it
created_at	when

Input: Request context and active rules

Output: Block or pass

Feature 6: When the infrastructure falls

Redis down? Kafka down? Each scope has a predefined answer: carry on or block.

How it works

- Every scope has a policy: fail open or fail closed.
- A dangerous scope (code execution, say) fails closed. A general one fails open with an alert.
- Every gateway keeps the last state it knew on disk.
- No fresh update for over five minutes? It switches to the safe policy automatically.

What it defends against

Threat: The control centre is knocked down to blind the mechanism.

Answer: Silence counts as a warning sign, and the gateway moves to the safe state (Dead Man's Switch).

Technologies and design decisions

Component	Type	Role
Chaos Monkey / Gremlin	Off the shelf	Break things on purpose to test
SQLite	Off the shelf	A local state copy for each gateway
Dead Man's Switch	Off the shelf	Missing heartbeats are an alarm
A separate policy per scope	Our design decision	Safety without hurting healthy service

Data structure: LocalStateSnapshot

Field	Meaning
scope_id	the scope
status	ACTIVE or SUSPENDED
last_updated	last update
fail_safe_policy	FAIL_OPEN or FAIL_CLOSED

Input: Heartbeats and a policy per scope

Output: An enforcement state even without infrastructure

Feature 7: Guarding the guard

The mechanism watches itself too: who signed, how many, when, and it alerts on anything unusual.

How it works

- Each key has a normal pattern: commands per hour, usual hours, usual scopes.

- Every command is compared with the pattern.
- A deviation of more than 3 standard deviations raises an alert.
- The alert flows into the security system the team already watches daily.

What it defends against

Threat: A valid key has fallen into the wrong hands and sends commands at odd hours.

Answer: The pattern deviates, and the alert goes out.

Technologies and design decisions

Component	Type	Role
Splunk / Elastic Security	Off the shelf	Alerts on the screen the team already knows
UEBA	Off the shelf	Who does what, when, and how it differs
Deterministic z-score, no AI	Our design decision	A security component must be transparent and explainable

Data structure: SignerBaseline

Field	Meaning
key_id	the key
commands_per_hour_avg / stddev	average and standard deviation
typical_active_hours	usual active hours
typical_scopes	usual scopes

Input: The command history of each key

Output: A deviation score, and an alert

Feature 8: Many regions

The command must reach every region where the model runs, even when links between regions break.

How it works

- Each region has its own Kafka cluster and its own fleet of gateways.
- Commands are replicated across regions in the same order.
- If the link breaks, each region carries on with the last command it received.
- When the link returns, everything syncs.

What it defends against

Threat: A cut between regions, or a whole data centre going down.

Answer: Each region keeps running, and syncs back later.

Technologies and design decisions

Component	Type	Role
Cluster Linking / MirrorMaker 2	Off the shelf	Replication between clusters

Component	Type	Role
Anycast	Off the shelf	One address, the nearest available
A visible lag metric for monitoring	Assembled	See how far a region lags

Data structure: RegionClusterStatus

Field	Meaning
region_id	the region
kafka_replication_lag_ms	how many ms behind
last_synced_command_id	the last synced command

Input: Commands from the centre

Output: A synced state in every region

Feature 9: Developer libraries

Connecting the mechanism to an existing system takes about ten lines of code.

How it works

- One API specification (OpenAPI) is the source of truth.
- Libraries are generated automatically for every language.
- Between gateway and control centre: gRPC, fast and strictly typed.
- Ready middleware for FastAPI and Express servers.

What mistake it prevents

Threat: Adoption that requires rewriting existing code, so nobody adopts.

Answer: Connect in about ten lines, without touching existing code.

Technologies and design decisions

Component	Type	Role
OpenAPI Generator	Off the shelf	Libraries in over 40 languages
gRPC + Protobuf	Off the shelf	Fast internal traffic
FastAPI	Off the shelf	Ready middleware
Express	Off the shelf	Ready middleware

Data structure: OpenAPISpec + ProtobufSchema

Field	Meaning
OpenAPISpec	a single source of truth for the API
ProtobufSchema	a type contract for internal traffic

Input: One specification

Output: Libraries for every language, and middleware

Feature 10: Simulate before stopping

Before signing a real stop, you see exactly what it would hit.

How it works

- A third action, DRY_RUN, beside STOP and RESUME.
- The gateway runs the same matching against real traffic.
- But it blocks nothing.
- It returns a report: how many requests, from which clients and regions.

What mistake it prevents

Threat: A region:eu-* that accidentally catches something unintended.

Answer: The exact blast radius is visible before signing.

Technologies and design decisions

Component	Type	Role
Traffic Mirroring	Off the shelf	Test on the real thing without touching it
The DRY_RUN action	Our design decision	Simulation as part of the mechanism itself

Data structure: commands.action

Field	Meaning
action	STOP / RESUME / DRY_RUN

Input: A proposed scope

Output: An impact report, with no blocking

Feature 11: Automatic expiry

A forgotten stop does not last forever: after four hours it lifts by itself, unless re-signed.

How it works

- Every stop command has expires_at, for example 4 hours.
- No renewal? The state returns to ACTIVE.
- Renewal needs a new, signed command.
- Every renewal, or its absence, stays in the audit log.

What mistake it prevents

Threat: A stop forgotten permanently because someone meant to renew and forgot.

Answer: Expiry is automatic, and renewal leaves a trace.

Technologies and design decisions

Component	Type	Role
Temporal.io	Off the shelf	The timer survives a server crash

Component	Type	Role
Expiry by default	Our design decision	Forgetting never becomes permanent damage

Data structure: scope_states.expires_at

Field	Meaning
expires_at	when the stop expires

Input: A stop command with an expiry

Output: An automatic return to ACTIVE

Six decisions that could have gone the other way

Ed25519 or RSA? We chose Ed25519. Verification is far faster and the signature is small, 64 bytes. When every millisecond at the gateway counts, that decides it. RSA stays out.

One person or a committee? We chose a committee, 3 of 5. One exposed key hands over full control, and one corrupt person is not stopped. A committee answers "who guards the guards". The price: coordination between signers.

Carry on or block when it all fails? We chose: it depends on the scope. A dangerous scope fails closed, a general one fails open with an alert. A safety mechanism should not take down a healthy service. The price: a policy per scope.

AI or statistics? We chose simple statistics. A critical security component must be deterministic, transparent and explainable. A z-score against a normal pattern does the job. The price: less "smart".

Cut or drain? We chose to drain. Requests already running finish within 10 seconds. A partial, misleading answer is worse than a short wait. The price: not instantly total.

Stop forever or for a while? We chose a stop with an expiry. Forgotten shutdowns are a known operational failure. Automatic expiry, and renewal by a new signature, leave a trace. The price: you must remember to renew.

What the POC taught us

In the POC	In the spec
A single key	A committee, 3 of 5
State in one process's memory	Redis and Kafka, across regions
One general stop	Targeted stop by scope
Failure was not handled	A policy per scope, and a Dead Man's Switch
A hash-chained log	A transparent public log, and anomaly monitoring